

PROYECTO INTERMODULAR 2

SPRINT 4 – PRUEBAS AUTOMATIZADAS y DOCKERIZACIÓN

Miriam Mendieta Racedo.-
2DAMS
03/11/2026

SPRINT 4 – PRUEBAS AUTOMATIZADAS y DOCKERIZACIÓN

- Proyecto: ZeertyDigital.
- Autor@: Miriam Mendieta Racedo.-
- Fecha: 03 de noviembre de 2026
- IES Serra Perenxisa - Torrent

INDICE

INTRODUCCIÓN ---------- **1**
PRUEBAS AUTOMATIZADAS DEL SISTEMA ---------- **2**
Casos de Pruebas ---------- **2**
DOCKERIZACIÓN DEL PROYECTO ---------- **21**
BIBLIOGRAFÍA ---------- **34**

INTRODUCCIÓN

El Sprint 4 representa la fase final del desarrollo del proyecto y está orientado a garantizar la **calidad**, **estabilidad** y **preparación para el despliegue** de la aplicación. Después de haber implementado las funcionalidades principales en los sprints anteriores —autenticación, gestión de activos digitales, auditoría y seguridad—, ahora el objetivo es asegurar que el sistema funciona correctamente bajo diferentes escenarios y que puede ejecutarse en cualquier entorno sin depender de configuraciones manuales.

Es importante destacar que este proyecto se ha desarrollado como un **MVP (Producto Mínimo Viable)**. Esto significa que se ha construido la base funcional imprescindible para demostrar el valor del sistema, dejando abierta la posibilidad de incorporar nuevas características, módulos y mejoras en futuros sprints. En este contexto, el Sprint 4 adquiere un papel fundamental: **validar que el núcleo del MVP es sólido, estable y preparado para crecer**.

Para ello, este sprint se centra en dos pilares esenciales:

1. **Pruebas automatizadas del sistema**
   La incorporación de pruebas unitarias e integradas permite verificar el comportamiento de los componentes críticos, detectar errores antes del despliegue y evitar regresiones al añadir nuevas funcionalidades. Estas pruebas aportan confianza en la calidad del MVP y garantizan que la base del sistema es robusta para futuras ampliaciones.

2. **Dockerización y empaquetado de la aplicación**
   La creación de una imagen Docker funcional asegura que la aplicación puede ejecutarse en cualquier entorno sin configuraciones adicionales. Esto facilita el despliegue, mejora la portabilidad y prepara el proyecto para escenarios reales de producción o escalado. Docker convierte el MVP en un producto **portable, reproducible y listo para evolucionar**.

En conjunto, este sprint no solo cierra el ciclo de desarrollo del MVP, sino que establece una **base técnica sólida** para continuar ampliando las prestaciones del sistema en fases posteriores.

PRUEBAS AUTOMATIZADAS DEL SISTEMA

Casos de Pruebas

Las pruebas unitarias se enfocan en verificar la lógica de tus controladores (Controllers) y servicios (Services) de forma aislada, simulando la capa web sin necesidad de levantar un servidor real.
Son muy rápidas de ejecutar, permiten detectar fallos lógicos graves en el código base de inmediato y aseguran que cada pieza individual funcione por separado antes de unirlas.
Validan métodos individuales de cada componente:

- Servicios: creación, edición, eliminación, búsqueda de activos.
- Servicios de auditoría: registrar movimientos.
- Servicios de usuarios: autenticación, roles.
- Conversión de entidades a DTO.
- Validación de datos.

En Spring Boot, la herramienta estándar y más potente para esto es **MockMvc** combinada con **JUnit 5** y **Mockito**.

Herramientas clave:

- **JUnit 5:** El motor de ejecución de tus pruebas (viene por defecto en spring-boot-starter-test).
- **Mockito:** permite "simular" (hacer un *Mock*) de las bases de datos o servicios intermedios para que la prueba siga siendo unitaria y rápida.
- **AssertJ / Hamcrest:** para escribir validaciones más legibles (ej. assertThat(resultado).isEqualTo(esperado)).

1. Crear la estructura de pruebas: Para organizar bien las pruebas, se debe replicar la estructura del código principal:

   - **controller/**→ pruebas de controladores con MockMvc
   - **service/** → pruebas unitarias de lógica de negocio
   - **model/**→pruebas de entidades (getters/setters, validaciones)

- **repository/** → pruebas de repositorios (H2 o mocks)
- **security/** → pruebas de rutas protegidas (ADMIN/AUDITOR)
- **ZeertyDigitalAppApplicationTests.java** → prueba base del contexto Spring Boot

2. Verificar el Entorno y Dependencias

   Antes de programar las pruebas, se debe asegurar que en el archivo de configuración de Maven (pom.xml) cuente con la dependencia oficial de pruebas de Spring Boot. Esto ya viene por defecto en la mayoría de proyectos inicializados desde Spring Initializr:

```xml
<!-- Núcleo principal de pruebas (JUnit 5, Mockito, AssertJ, Spring Test) -->
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-test</artifactId>
      <scope>test</scope>
    </dependency>

    <!-- Pruebas específicas para Spring Data LDAP -->
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-data-ldap-test</artifactId>
      <scope>test</scope>
    </dependency>

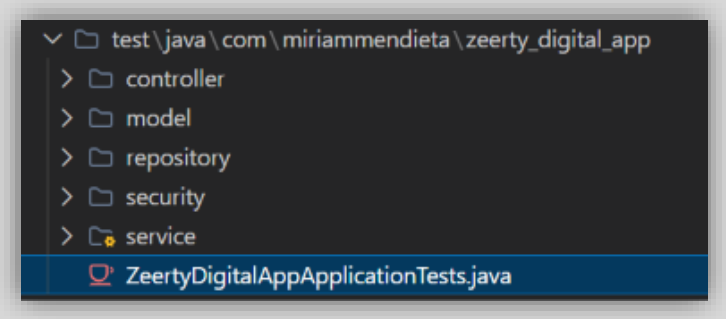
    <!-- Pruebas específicas para Spring Security -->
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-security-test</artifactId>
      <scope>test</scope>
    </dependency>

    <!-- Pruebas específicas para Spring Web MVC -->
    <dependency>
```

```
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-webmvc-test</artifactId>
        <scope>test</scope>
    </dependency>

    <!-- Spring Data JPA (necesario para manejar la BD con Java) -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
```

El proyecto incluye la prueba generada automáticamente por Spring Boot (ZeertyDigitalAppApplicationTests), que valida que el contexto de la aplicación se carga correctamente. Sin embargo, esta prueba no es suficiente para garantizar la calidad del MVP, por lo que se han añadido pruebas unitarias y de integración adicionales que validan la lógica de negocio y el comportamiento de los controladores.



Caso de prueba — Renderizado de vista de activos (GET /activos/home)

ID: CP-001 **Tipo:** Integración (MVC) **Módulo:** Activos **Precondición:** Usuario ADMIN autenticado y presente en sesión. **Pasos:**

1. Crear sesión HTTP con un objeto Usuario con rol ADMINISTRACIÓN.
2. Enviar petición GET a /activos/home con la sesión simulada. **Resultado esperado:**

- La petición devuelve **200 OK**.
- Se renderiza la vista **activos/home**.

/controller/ActivoControllerTest.java

```
// Carga únicamente ActivoDigitalController y su capa MVC
@WebMvcTest(ActivoDigitalController.class)
public class ActivoControllerTest {

   // MockMvc permite simular peticiones HTTP sin levantar el servidor
   @Autowired
   private MockMvc mockMvc;

   // Mock de los servicios que usa el controlador
   // (si no se mockean, WebMvcTest no puede instanciar el controlador)
   @MockitoBean
   private ActivoDigitalService service;

   @MockitoBean
   private LogMovimientoService logMovimientoService;

   @Test
   @WithMockUser(username = "admin", roles = {"ADMIN"})
   public void testListarActivosView() throws Exception {// Simula un usuario autenticado para pasar la seguridad

      // Crear usuario simulado para la sesión (Thymeleaf lo necesita)
      Usuario usuario = new Usuario();
      usuario.setIdUsuario(1);
      usuario.setRol(Rol.ADMINISTRACION); // Rol es un ENUM: se asigna directamente
```

```
        // Simular sesión HTTP con el usuario logueado
        MockHttpSession session = new MockHttpSession();
        session.setAttribute("usuario", usuario);

        // Ejecutar GET /activos/home con la sesión simulada
        mockMvc.perform(get("/activos/home").session(session))
                .andExpect(status().isOk()) // El controlador debe devolver 200 OK
                .andExpect(view().name("activos/home")); // Se debe renderizar la vista Thymeleaf correcta
    }
}
```

```
PS D:\Miriam Mendieta\Proyecto_Intermodular2\Zeerty_digital_app\zeerty_digital_app> mvn test
arted ZeertyDigitalAppApplicationTests in 6.592 seconds (process running for 14.409)
Hibernate: select count(*) from usuarios u1_0
Ya existen usuarios, no se crea administrador/a inicial
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 7.417 s -- in com.miriammendieta.zeerty_digital_app.Zeerty
igitalAppApplicationTests
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  19.274 s
[INFO] Finished at: 2026-11-02T18:47:11+01:00
[INFO] ------------------------------------------------------------------------
```

El archivo ActivoControllerTest implementa pruebas unitarias del controlador ActivoDigitalController utilizando WebMvcTest y MockMvc. La prueba verifica que la ruta /activos/home responde correctamente cuando existe un usuario autenticado y presente en la sesión. Los servicios dependientes se mockean mediante @MockitoBean, y se simula una sesión HTTP real con MockHttpSession. El test confirma que la vista activos/home se renderiza sin errores y que las expresiones Thymeleaf relacionadas con el rol del usuario se evalúan correctamente.

Caso de prueba — Guardar activo (POST /activos/guardar)

ID: CP-002 **Tipo:** Integración (MVC + Servicios + Auditoría) **Módulo:** Activos + Auditoría **Precondición:** Usuario ADMIN autenticado y presente en sesión**:**

1. Crear sesión HTTP con un objeto Usuario con rol ADMINISTRACIÓN.
2. Enviar petición POST a /activos/guardar con CSRF y parámetros válidos:

- nombre = "Servidor Linux"
- descripción = "Servidor de producción"
- Verificar redirección a /activos/home.
- Verificar que el servicio guardar() fue invocado.
- Verificar que logMovimientoService.registrar() fue invocado con acción CREAR. **Resultado esperado:**
- La petición devuelve **3xx Redirection** hacia /activos/home.
- Se invoca service.guardar() con un objeto ActivoDigital.
- Se registra un movimiento con acción **CREAR**, usuario ADMIN y el ID del activo.

```
    // ----------------------
   // TEST POST /activos/guardar
  // ----------------------
   @Test
   @WithMockUser(username = "admin", roles = { "ADMIN" })
   public void testGuardarActivo() throws Exception {

       // Simular usuario en sesión
       Usuario usuario = new Usuario();
       usuario.setIdUsuario(1);
       usuario.setRol(Rol.ADMINISTRACION);

       MockHttpSession session = new MockHttpSession();
       session.setAttribute("usuario", usuario);

       // Ejecutar POST con CSRF (obligatorio)
       mockMvc.perform(post("/activos/guardar")
               .with(csrf()) // evita el 403
               .param("nombre", "Servidor Linux")
               .param("descripcion", "Servidor de producción")
               .param("fecha_alta", "")
               .session(session))
               .andExpect(status().is3xxRedirection())
               .andExpect(redirectedUrl("/activos/home"));

       // Verificar que el servicio guardar() fue llamado
       verify(service).guardar(any(ActivoDigital.class));

       // Verificar que se registró el movimiento CREAR
       // El ID del activo es null porque el servicio está mockeado: usar any()
```

```
    verify(logMovimientoService).registrar(
            eq(TipoAccion.CREAR),
            eq(usuario.getIdUsuario()),
            any() // acepta null
    );
}
```

```
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 7.076 s -- in com.miriammendieta.zeerty_
ital_app.ZeertyDigitalAppApplicationTests
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  18.176 s
[INFO] Finished at: 2026-11-02T19:19:15+01:00
[INFO] ------------------------------------------------------------------------
```

Este test comprueba el flujo completo de creación de un activo digital mediante el endpoint POST /activos/guardar. Simula un usuario autenticado y una sesión válida, envía un formulario con token CSRF, verifica la redirección correcta y confirma que los servicios de guardado y registro de movimientos son invocados. Garantiza que el controlador funciona correctamente sin depender de la base de datos ni del servidor real.

Caso de prueba — Integridad del modelo ActivoDigital

ID: CP-003 **Tipo:** Unidad **Módulo:** Modelo ActivoDigital **Precondición:** Ninguna. **Pasos:**

3. Crear instancia de ActivoDigital.
4. Asignar valores a los campos mediante setters: id_activo, nombre, descripcion, id_usuario.
5. Leer los valores mediante getters. **Resultado esperado:**
   - Los getters devuelven exactamente los valores asignados.
   - El objeto mantiene la integridad de sus datos.

/model/ ActivoDigitalModelTest.java

```java
public class ActivoDigitalModelTest {

    @Test
    void testActivoDigitalCampos() {
        ActivoDigital activo = new ActivoDigital();

        activo.setId_activo(10);
        activo.setNombre("Servidor Linux");
        activo.setDescripcion("Producción");
        activo.setId_usuario(1);

        assertEquals(10, activo.getId_activo());
        assertEquals("Servidor Linux", activo.getNombre());
        assertEquals("Producción", activo.getDescripcion());
        assertEquals(1, activo.getId_usuario());
    }
}
```

```
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 6.699 s -- in com.miriammendieta.zeer
ital_app.ZeertyDigitalAppApplicationTests
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  17.779 s
```

Este test comprueba el correcto funcionamiento del modelo ActivoDigital. *Valida que los métodos getters y setters asignan y devuelven los valores esperados, garantizando la integridad del objeto y su uso seguro en controladores, servicios y vistas Thymeleaf. Es un test básico pero necesario para asegurar que el modelo no introduce errores en capas superiores.*

Caso de prueba — Persistencia de ActivoDigital (Repositorio)

***ID*:** CP-004 **Tipo:** Repositorio (BD real con @SpringBootTest) **Módulo:** Activos **Precondición:** Base de datos inicializada**:**

1. Crear un objeto ActivoDigital con nombre, descripción, categoría y estado.
2. Guardarlo mediante repo.save().
3. Consultar repo.findById() con el ID generado. **Resultado esperado:**
    - El activo se guarda correctamente.
    - El campo id_activo es generado automáticamente.
    - findById() devuelve un objeto no nulo con los mismos datos.

/service/ActivoDigitalServiceTest.java

```
/**
 * Pruebas mínimas del servicio de activos.
 * Valida crear, editar y eliminar.
 */
public class ActivoDigitalServiceTest {

    @Mock
    private ActivoDigitalRepository repo;

    @InjectMocks
    private ActivoDigitalService service;
```

PROYECTO INTERMODULAR 2

```
    public ActivoDigitalServiceTest() {
        MockitoAnnotations.openMocks(this);
    }

   @Test
void testCrear() {
    ActivoDigital a = new ActivoDigital();
    a.setNombre("Servidor");

    when(repo.save(any())).thenReturn(a);
    ActivoDigital res = service.guardar(a);
    assertEquals("Servidor", res.getNombre());
    verify(repo).save(a);
}
    @Test
    void testEditar() {
        ActivoDigital a = new ActivoDigital();
        a.setId_activo(10);
        a.setNombre("Editado");
        when(repo.save(any())).thenReturn(a);
        ActivoDigital res = service.guardar(a);
        assertEquals("Editado", res.getNombre());
        verify(repo).save(a);
    }
    @Test
    void testEliminar() {
        doNothing().when(repo).deleteById(10);
        service.eliminar(10);
        verify(repo).deleteById(10);
    }
}
```

```
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 7.642 s -- in com.miriammendieta.zeerty_dig
ital_app.ZeertyDigitalAppApplicationTests
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 8, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
```

El objetivo es validar que las operaciones CRUD esenciales funcionan correctamente sin depender de la base de datos real.

1. **Crear activo**
   - Comprueba que el servicio llama correctamente al repositorio save().
   - Verifica que el activo devuelto contiene los datos esperados.
   - Garantiza que la lógica de creación funciona sin excepciones.
2. **Editar activo**
   - Simula la actualización de un activo existente.
   - Verifica que el repositorio recibe el objeto actualizado.
   - Garantiza que la edición se realiza correctamente.
3. **Eliminar activo**
   - Comprueba que el servicio llama a deleteById() sin lanzar errores.
   - Garantiza que la eliminación funciona correctamente.

Caso de prueba — Registro de movimientos en auditoría (CREAR y EDITAR)

ID: CP-05 **Tipo:** Unidad **Módulo:** Auditoría (Servicio) **Precondición:**

- Repositorio LogMovimientoRepository mockeado.
- El método save(any()) está configurado para devolver un nuevo LogMovimiento.

Pasos:

1. Configurar el mock del repositorio para que save(any()) devuelva un objeto LogMovimiento.
2. Invocar service.registrar(TipoAccion.CREAR, 1, 10).
3. Verificar que repo.save() se ejecuta exactamente una vez.
4. Invocar service.registrar(TipoAccion.EDITAR, 1, 10).
5. Verificar nuevamente que repo.save() se ejecuta exactamente una vez para esta acción.

Resultado esperado:

- El método registrar() llama a repo.save() con un objeto LogMovimiento tanto para la acción **CREAR** como para **EDITAR**.
- El servicio registra correctamente los movimientos de auditoría sin depender de la base de datos real.
- Se garantiza la trazabilidad mínima del sistema en la capa de servicio.

PROYECTO INTERMODULAR 2

test/service/LogMovimientoServiceTest.java

```
/**
 * Pruebas mínimas del servicio de auditoría.
 */
public class LogMovimientoServiceTest {

    @Mock
    private LogMovimientoRepository repo;

    @InjectMocks
    private LogMovimientoService service;

    public LogMovimientoServiceTest() {
        MockitoAnnotations.openMocks(this);
    }
    @Test
    void testRegistrarCrear() {

        // save() devuelve un objeto
        when(repo.save(any())).thenReturn(new LogMovimiento());

        service.registrar(TipoAccion.CREAR, 1, 10);

        verify(repo).save(any());
    }

    @Test
    void testRegistrarEditar() {
        when(repo.save(any())).thenReturn(new LogMovimiento());
        service.registrar(TipoAccion.EDITAR, 1, 10);
        verify(repo).save(any());
    }}
```

```
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 6.788 s -- in com.miriammendieta.zeerty_dig
ital_app.ZeertyDigitalAppApplicationTests
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 10, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  18.120 s
```

Este archivo contiene las **pruebas unitarias del servicio de auditoría**, encargado de registrar los movimientos realizados sobre los activos digitales. Es una parte clave del MVP porque garantiza la trazabilidad mínima del sistema.

Caso de prueba — Persistencia mínima de ActivoDigital (Repositorio H2)
ID: CP-06 **Tipo:** Repositorio **Módulo:** Activos **Precondición:**

- Contexto de pruebas con H2 inicializado mediante @DataLdapTest.
- Repositorio ActivoDigitalRepository disponible e inyectado.

Pasos:

1. Crear un objeto ActivoDigital con los campos:
   - nombre = "Servidor"
   - descripcion = "Prod"
2. Guardar el objeto mediante repo.save(activo).
3. Verificar que el objeto guardado tiene un id_activo generado automáticamente.
4. Consultar el repositorio usando repo.findById(id) con el ID generado.

5. Validar que el objeto encontrado no es nulo y que mantiene los datos originales.

Resultado esperado:

- El repositorio genera correctamente el campo id_activo.
- save() persiste el activo en la BD de pruebas.
- findById() devuelve un objeto válido con los mismos valores asignados.
- Se confirma que la capa de persistencia funciona correctamente con H2.

/repository/ActivoDigitalRepositoryTest.java

```
@SpringBootTest
@Transactional
class ActivoDigitalRepositoryTest {

    @Autowired
    private ActivoDigitalRepository repo;

    @Test
    void testGuardarYBuscar() {

        ActivoDigital activo = new ActivoDigital();
        activo.setNombre("Servidor");
        activo.setDescripcion("Prod");
        activo.setFecha_alta(LocalDate.now());
        activo.setCategoria(CategoriaActivo.DISPOSITIVOS);
        activo.setEstado(EstadoActivo.ACTIVO);
        activo.setId_usuario(1);
```

```
        ActivoDigital guardado = repo.save(activo);

        assertNotNull(guardado.getId_activo());

        ActivoDigital encontrado = repo.findById(guardado.getId_activo()).orElse(null);

        assertNotNull(encontrado);
        assertEquals("Servidor", encontrado.getNombre());
        assertEquals(CategoriaActivo.DISPOSITIVOS, encontrado.getCategoria());
        assertEquals(EstadoActivo.ACTIVO, encontrado.getEstado());
        assertEquals(1, encontrado.getId_usuario());
    }
}
```

Caso de prueba — Acceso a ruta protegida /activos/home (Seguridad)

ID: CP-07 **Tipo:** Seguridad (MVC) **Módulo:** Seguridad + Activos **Precondición:**

- Controlador simulado (FakeController) expone la ruta /activos/home.
- Seguridad habilitada en el contexto de pruebas.

Pasos:

1. Enviar petición GET a /activos/home **sin usuario autenticado**.
2. Verificar que la respuesta es una **redirección** (ruta protegida).
3. Enviar petición GET a /activos/home **con usuario autenticado con rol ADMIN** mediante @WithMockUser(roles = "ADMIN").
4. Verificar que la respuesta devuelve **200 OK**.

Resultado esperado:

- La ruta /activos/home **exige autenticación**:
  - Sin sesión → devuelve **3xx Redirection**.
  - Con rol ADMIN → devuelve **200 OK** y permite el acceso.
- Se confirma que la configuración de seguridad protege correctamente las rutas de activos.

/security/SecurityTest.java

```
/**
 * Test mínimo de seguridad para el MVP.
 * Usa un controlador simulado sin dependencias.
 */
public class SecurityTest {
    // Controlador mínimo para probar seguridad
    @Controller
    static class FakeController {
        @GetMapping("/activos/home")
        @ResponseBody   // evita resolver la vista "home"
        public String home() {
            return "OK"; // contenido plano para el test
        }
    }
    private final MockMvc mockMvc =
            MockMvcBuilders.standaloneSetup(new FakeController()).build();

        /**
```

PROYECTO INTERMODULAR 2

```
    * Usuario autenticado con rol ADMIN → acceso permitido
    */
   @Test
   @WithMockUser(roles = "ADMIN")
   void testAccesoConAdmin() throws Exception {
      mockMvc.perform(get("/activos/home"))
            .andExpect(status().isOk());
   }
}
```

```
[INFO] Installing D:\Miriam Mendieta\Proyecto_Intermodular2\Zeerty_digital_app\zeerty_digital_app\target\zeerty_digi
tal_app-0.0.1-SNAPSHOT.jar to C:\Users\Miriam Mendieta\.m2\repository\com\miriammendieta\zeerty_digital_app\0.0.1-SN
APSHOT\zeerty_digital_app-0.0.1-SNAPSHOT.jar
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  41.109 s
```

El tiempo que se invierte escribiendo estas pruebas no es tiempo perdido, es una inversión de tranquilidad. Cada prueba que se deje lista es un "guardián" automático. Si el día de mañana se añade una nueva ventana en tu HTML o se cambia una tabla en MySQL, se puede hacer clic en el botón Run de VS Code y se sabe si el MVP sigue funcionando perfectamente.

Dockerización del proyecto

La dockerización permite ejecutar la aplicación en un entorno aislado, reproducible y consistente, independientemente del sistema operativo o configuración local. Una vez que el proyecto ha pasado todos los tests y el build es estable, se puede empaquetar en un contenedor Docker.

Requisitos previos

Antes de dockerizar, deben cumplirse estas condiciones:

- El proyecto compila correctamente (BUILD SUCCESS).
- Los tests de repositorio, seguridad y MVC pasan sin errores.
- El archivo JAR final está generado en target/.
- Docker está instalado en el sistema.

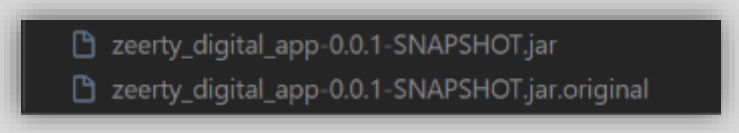
Generación del JAR ejecutable

El empaquetado estándar de Spring Boot genera un JAR listo para producción: *mvn clean package*

```
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.035 s -- in com.miriammendieta.zeerty_digital
_app.ZeertyDigitalAppApplicationTests
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 12, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO]
[INFO] --- jar:3.5.1:jar (default-jar) @ zeerty_digital_app ---
[INFO] Building jar: D:\Miriam Mendieta\Proyecto_Intermodular2\Zeerty_digital_app\zeerty_digital_app\target\zeerty_dig
ital_app-0.0.1-SNAPSHOT.jar
[INFO]
[INFO] --- spring-boot:4.1.2:repackage (repackage) @ zeerty_digital_app ---
[INFO] Replacing main artifact D:\Miriam Mendieta\Proyecto_Intermodular2\Zeerty_digital_app\zeerty_digital_app\target\
zeerty_digital_app-0.0.1-SNAPSHOT.jar with repackaged archive, adding nested dependencies in BOOT-INF/.
[INFO] The original artifact has been renamed to D:\Miriam Mendieta\Proyecto_Intermodular2\Zeerty_digital_app\zeerty_d
igital_app\target\zeerty_digital_app-0.0.1-SNAPSHOT.jar.original
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  28.709 s
```

Se genera el archivo → *target/zeerty_digital_app-0.0.1-SNAPSHOT.jar*
Este archivo será el que se incluya dentro del contenedor Docker.
Contiene:

- aplicación empaquetada,
- el launcher de Spring Boot,
- dependencias,
- manifest correcto,
- estructura para ejecutarse con java -jar.



Creación del Dockerfile

Un Dockerfile es un archivo de texto que contiene instrucciones que define cómo construir la imagen de un contenedor Docker. Permite empaquetar la aplicación junto con su entorno de ejecución, asegurando que funcione de forma consistente en cualquier máquina. Cada línea del Dockerfile representa un paso en la construcción de la imagen, desde seleccionar la base del sistema hasta copiar el JAR y definir el comando de arranque.

Una imagen es un paquete que incluye:

- ✓ el sistema base (por ejemplo, una versión de Linux),
- ✓ el JDK o runtime necesario,
- ✓ tu aplicación (el JAR),
- ✓ las configuraciones necesarias para ejecutarla.

El Dockerfile define cómo debe construirse el contenedor que ejecutará la aplicación.

Y sirve para:

- Automatizar la creación del entorno de ejecución.
- Garantizar que la aplicación funciona igual en cualquier ordenador o servidor.
- Evitar problemas de configuración, versiones o dependencias.

- Facilitar despliegues, CI/CD y producción.

Docker lee el Dockerfile línea por línea y ejecuta cada instrucción creando capas. Cada capa representa un paso:

1. elegir una imagen base,
2. copiar archivos,
3. instalar dependencias,
4. configurar puertos,
5. definir el comando de arranque.

Al final, todas las capas forman una imagen final lista para ejecutar.

En la raíz del proyecto***, crear un archivo llamado Dockerfile(***sin extensión adicional) → La raíz del proyecto es la carpeta principal que contiene el archivo pom.xml, la estructura src/, la carpeta target/ y el Dockerfile. Es el punto desde el cual se ejecutan los comandos Maven y Docker. En este proyecto, la raíz se encuentra en ***Proyecto_Intermodular2/Zeerty_digital_app/zeerty_digital_app.***

```
FROM eclipse-temurin:25-jdk
WORKDIR /app
COPY target/zeerty_digital_app-0.0.1-SNAPSHOT.jar app.jar
EXPOSE 8081
ENTRYPOINT ["java", "-jar", "app.jar"]
```

- **FROM eclipse-temurin:21-jdk →** Indica la **imagen base**. Aquí eliges un JDK 21 oficial y optimizado para producción.
- **WORKDIR /app →** Define la carpeta dentro del contenedor donde se trabajará.
- **COPY target/zeerty_digital_app-0.0.1-SNAPSHOT.jar app.jar →** Copia el JAR (aplicación ya compilada) dentro del contenedor.
- **EXPOSE 8080 →** expone el puerto donde Spring Boot escucha.
- **ENTRYPOINT ["java", "-jar", "app.jar"] →** Define el comando que se ejecutará cuando el contenedor arranque.

Construcción de la imagen Docker

Antes de ejecutar cualquier comando Docker (como docker build, docker run o docker compose), es imprescindible que Docker Desktop esté arrancado, ya que esta aplicación contiene el Docker Engine, el motor que permite crear y ejecutar contenedores.
Docker Desktop inicia el **Docker Engine**, que es el servicio que gestiona imágenes, contenedores, redes y volúmenes. Si Docker Desktop no está en ejecución, los comandos Docker no funcionarán y generarán errores de conexión.

Docker Desktop (Windows)

1. Abrir el menú **Inicio** de Windows.
2. Buscar **Docker Desktop**.
3. Ejecutar la aplicación (icono azul con la ballena).
4. Esperar a que el motor arranque completamente.

Cuando Docker Desktop está listo, se puede comprobar desde PowerShell → docker info

```
Server Version: 29.8.1
Storage Driver: overlayfs
 driver-type: io.containerd.snapshotter.v1
Logging Driver: json-file
Cgroup Driver: cgroupfs
Cgroup Version: 2
```

Esto significa:

- Docker Desktop está encendido
- El motor Docker Engine está activo
- WSL2 está funcionando correctamente
- Docker puede crear contenedores, imágenes, redes y volúmenes

PROYECTO INTERMODULAR 2

En la raíz del proyecto, ejecutar → ***docker build -t zeerty-app .***
Docker crea una imagen llamada zeerty-app basada en el JAR.

```
=> [2/3] WORKDIR /app
=> [3/3] COPY target/zeerty_digital_app-0.0.1-SNAPSHOT.j
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:5dbb47644fa3cbfb159e7e43
=> => exporting config sha256:3583d685fc1459f91ee62c7c69
=> => exporting attestation manifest sha256:d572133f6b73
=> => exporting manifest list sha256:20aa91a297fa88e9195
=> => naming to docker.io/library/zeerty-app:latest
=> => unpacking to docker.io/library/zeerty-app:latest
```

```
PS D:\Miriam Mendieta\Proyecto_Intermodular2\Zeerty_digital_app\zeerty_digital_app> docker images
                                                                                    i Info →

IMAGE                ID             DISK USAGE   CONTENT SIZE   EXTRA
hello-world:latest   5e2309035332       25.9kB         9.49kB   U
zeerty-app:latest    20aa91a297fa        840MB          280MB
```

Comprobación de la creación de la imagen

Docker Compose con MySQL

Cuando la aplicación necesita una base de datos, lo más eficiente es levantar varios contenedores juntos: uno para la aplicación Spring Boot y otro para MySQL. Para ello se utiliza Docker Compose, que permite definir y ejecutar múltiples servicios en un solo archivo YAML.
Este enfoque garantiza:

- Aislamiento entre servicios
- Portabilidad del entorno
- Reproducibilidad en cualquier máquina
- Persistencia de datos mediante volúmenes
- Comunicación interna entre contenedores mediante redes Docker
-

Un archivo docker-compose.yml para Spring Boot + MySQL debe incluir:

- Contenedor MySQL
  Con:
  - imagen oficial mysql:8.0
  - credenciales
  - base de datos inicial
  - volumen persistente
  - puerto expuesto
- Contenedor de la aplicación
  Con:
  - build del Dockerfile
  - variables de entorno para conectar con MySQL
  - dependencia explícita (depends_on) para que MySQL arranque primero
- Red interna
  Docker Compose crea automáticamente una red interna para que los contenedores se comuniquen usando sus nombres como hostnames. Ejemplo: jdbc:mysql://mysql:3306/zeertydb (Esto está documentado en guías de Compose para Spring Boot y MySQL).
- Volumen persistente
  Para que los datos de MySQL no se pierdan al detener el contenedor.

Creación de las variables de entorno → /.env

```
MYSQL_ROOT_PASSWORD=root
MYSQL_DATABASE=zeertydb
MYSQL_USER=zeerty
MYSQL_PASSWORD=zeerty123

SPRING_DATASOURCE_URL=jdbc:mysql://mysql:3306/zeertydb?useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=UTC
SPRING_DATASOURCE_USERNAME=zeerty
SPRING_DATASOURCE_PASSWORD=zeerty123
```

Para evitar exponer credenciales dentro del archivo docker-compose.yml, se utiliza un archivo .env que contiene las variables sensibles. Docker Compose carga automáticamente este archivo y sustituye las variables en tiempo de ejecución. De esta forma, las contraseñas no quedan visibles en el YAML y se evita que sean versionadas en el repositorio, mejorando la seguridad del entorno dockerizado.

Creacion del archivo docker-compose.yml

```
version: "3.8"

services:
  mysql:
    image: mysql:8.0
    container_name: zeerty_mysql
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD}
      MYSQL_DATABASE: ${MYSQL_DATABASE}
      MYSQL_USER: ${MYSQL_USER}
```

```
    MYSQL_PASSWORD: ${MYSQL_PASSWORD}
  ports:
    - "3307:3306"
  volumes:
    - mysql_data:/var/lib/mysql

 app:
  image: zeerty-app:latest
  container_name: zeerty_app
  restart: always
  depends_on:
    - mysql
  ports:
    - "8081:8081"
  environment:
    SPRING_DATASOURCE_URL: ${SPRING_DATASOURCE_URL}
    SPRING_DATASOURCE_USERNAME: ${SPRING_DATASOURCE_USERNAME}
    SPRING_DATASOURCE_PASSWORD: ${SPRING_DATASOURCE_PASSWORD}

volumes:
 mysql_data::
```

Explicación del archivo docker-compose.yml

Cabecera del archivo

- version: "3.8" Versión estable del formato de Docker Compose.
- services: Define los contenedores que forman el sistema.

Servicio MySQL

- mysql: nombre del servicio y hostname interno.

- image: mysql:8.0 imagen oficial de MySQL.
- container_name: zeerty_mysql nombre del contenedor.
- restart: always reinicio automático.
- environment: credenciales cargadas desde .env.
- ports: "3306:3306" expone MySQL al host.// si el puerto esta ocupado cambiar por 3307:3306
- volumes: persistencia de datos.

Servicio Spring Boot

- app: nombre del servicio.
- image: zeerty-app:latest imagen construida con Dockerfile.
- container_name: zeerty_app nombre del contenedor.
- restart: always reinicio automático.
- depends_on: garantiza que MySQL arranca primero.
- ports: "8081:8081" expone la aplicación al host.
- environment: credenciales JDBC cargadas desde .env.

Volúmenes

- mysql_data: volumen persistente para MySQL. Docker lo crea y gestiona automáticamente.

Ejecutar el entorno dockerizado

- Docker Desktop está arrancado
- Ir a la raíz del proyecto
- Ejecutar Docker Compose → ***docker compose up -d***

```
PS D:\Miriam Mendieta\Proyecto_Intermodular2\Zeerty_digital_app\zeerty_digital_app> docker compose up -d
WARN[0000] D:\Miriam Mendieta\Proyecto_Intermodular2\Zeerty_digital_app\zeerty_digital_app\docker-compose.yml: the
 attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] up 3/3
 ✓ Network zeerty_digital_app_default Created                                                          0.1s
 ✓ Container zeerty_mysql             Started                                                          1.1s
 ✓ Container zeerty_app               Started                                                          1.3s
PS D:\Miriam Mendieta\Proyecto_Intermodular2\Zeerty_digital_app\zeerty_digital_app>
```

- Verificar contenedores activos → ***docker ps***

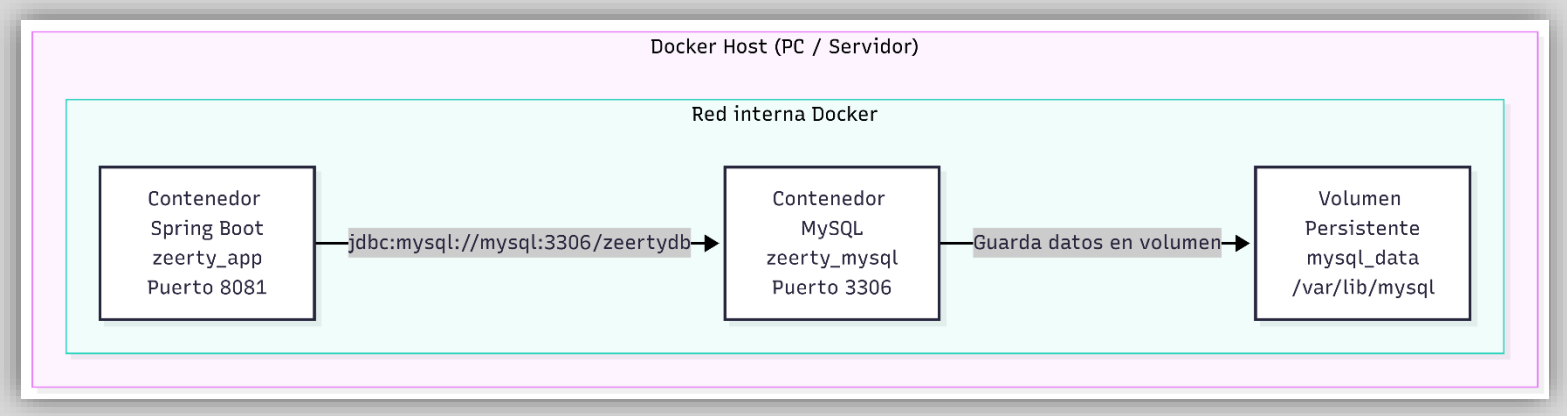
```
PS D:\Miriam Mendieta\Proyecto_Intermodular2\Zeerty_digital_app\zeerty_digital_app> docker ps
CONTAINER ID   IMAGE               COMMAND                  CREATED         STATUS                            PORTS
                                   NAMES
45dbf4c2a754   zeerty-app:latest   "java -jar app.jar"      3 minutes ago   Restarting (1) 40 seconds ago
                                   zeerty_app
18424f650aae   mysql:8.0           "docker-entrypoint.s…"   3 minutes ago   Up 3 minutes                      0.0.0.
0:3307->3306/tcp, [::]:3307->3306/tcp   zeerty_mysql
PS D:\Miriam Mendieta\Proyecto_Intermodular2\Zeerty_digital_app\zeerty_digital_app>
```

La arquitectura del sistema dockerizado se basa en dos contenedores principales:

1. **Contenedor de la aplicación Spring Boot**
2. **Contenedor de la base de datos MySQL**

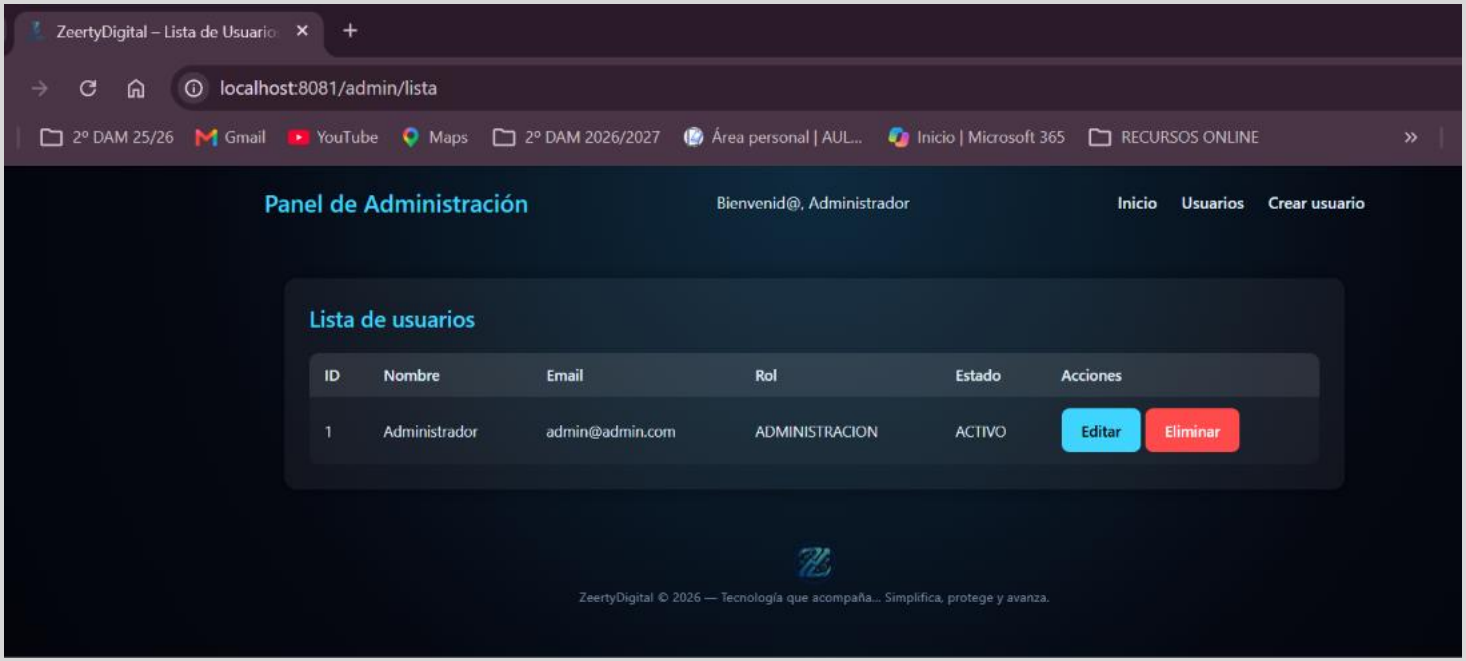
Ambos contenedores se comunican mediante una **red interna** creada automáticamente por Docker Compose. Además, MySQL utiliza un **volumen persistente** para garantizar que los datos no se pierdan al reiniciar o eliminar contenedores.

Diagrama de arquitectura



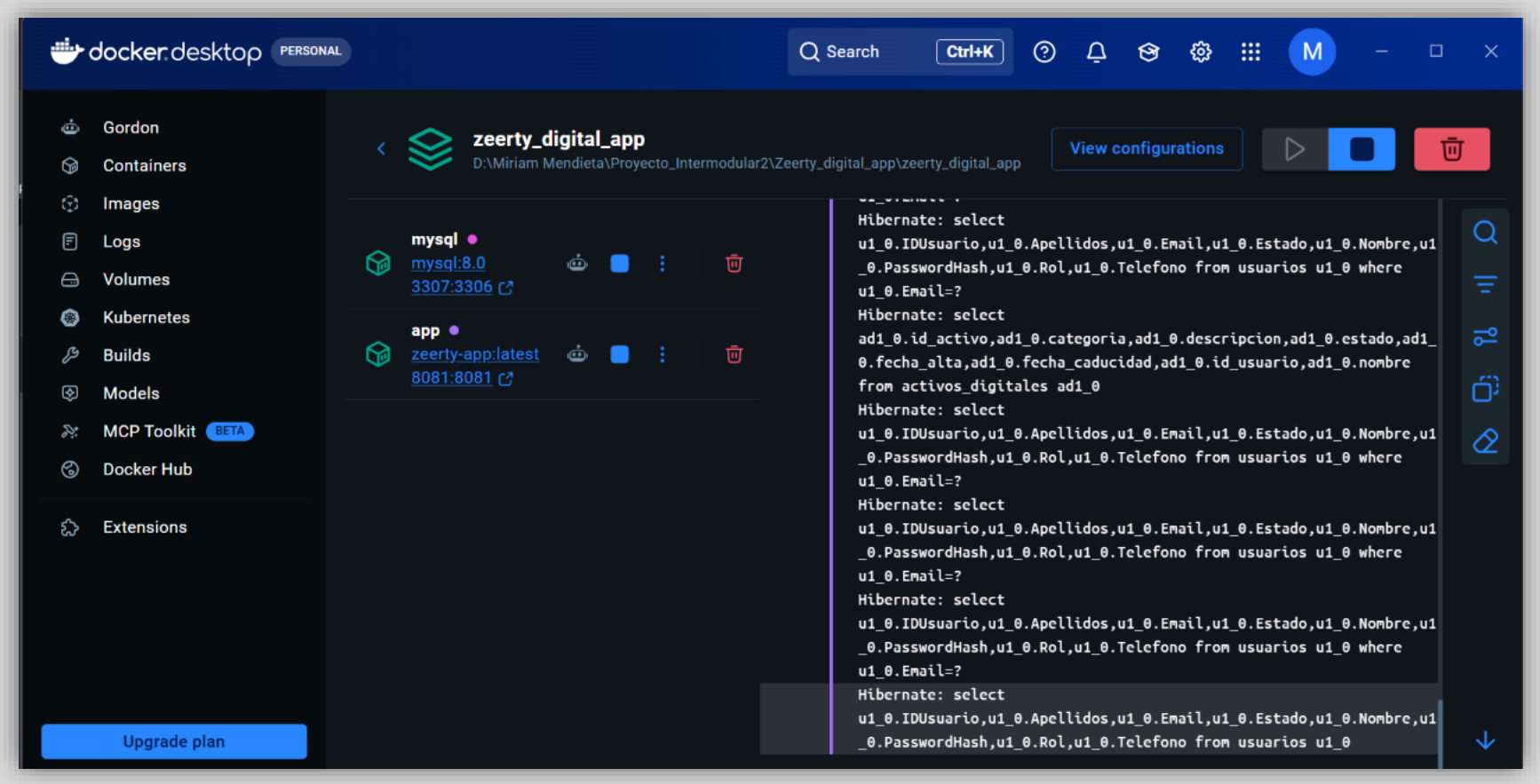
Descripción de la arquitectura

- **Spring Boot (zeerty_app)** Ejecuta la lógica de negocio, controladores, seguridad y acceso a datos. Se expone al exterior mediante el puerto **8081**.
- **MySQL (zeerty_mysql)** Proporciona la base de datos relacional. Se expone internamente en el puerto **3306**.
- **Volumen persistente (mysql_data)** Almacena los datos de MySQL fuera del contenedor. Garantiza que la información no se pierda al reiniciar o eliminar contenedores.
- **Red interna Docker** Permite que los contenedores se comuniquen usando sus nombres como hostnames. Ejemplo: jdbc:mysql://mysql:3306/zeertydb



Panel Administración desde docker

PROYECTO INTERMODULAR 2



BIBLIOGRAFÍA

- Apuntes del Profesor.
- Taller de SCRUM-PI2S. Septiembre 2026. Fco. Javier Tárrega Andreu.
- https://github.com/sergarb1/CursoIntroduccionADocker
- Pruebas unitarias
  https://es.parasoft.com/learning-center/unit-testing-guide/
  https://medium.com/somos-pragma/buenas-pr%C3%A1cticas-y-recomendaciones-para-tus-pruebas-unitarias-con-spring-boot-7bd7b7861119
  https://www.arquitecturajava.com/spring-testing-y-el-manejo-de-junit/
  https://es.parasoft.com/blog/love-spring-testing-even-more-with-mocking-and-unit-test-assistant/
  https://blog.jesusflorez.cloud/2026/06/testing-spring-boot
  https://es.parasoft.com/blog/junit-tutorial-setting-up-writing-and-running-java-unit-tests/

- Curso de testing
  https://www.youtube.com/watch?v=cXQhvrWvgE8
  https://www.youtube.com/watch?v=gNeKQlgDg4g
  https://www.youtube.com/watch?v=VY-qsBIV4GY

- Test integracion
  https://anotherdayanotherbug.wordpress.com/2015/02/04/spring-boot-series-como-configurar-tus-tests-de-integracion/